SET

Autonóm multi-agent orkesztrálás Claude Code-dal

Adj neki egy spec-et -- kapsz mergelt feature-öket.

2026. április

Az előadás menete

Idő	Téma
5 perc	**A probléma** -- Miért nem elég a "prompt and pray"?
5 perc	**A bemenet** -- Spec + Design = minőségi input
20 perc	**E2E Demo: MiniShop** -- A teljes pipeline végigvezetése
10 perc	**Quality Gates** -- Determinisztikus minőségbiztosítás
5 perc	**Dashboard & Monitoring** -- Valós idejű felügyelet
5 perc	**Architektúra** -- 3 réteg, plugin rendszer
5 perc	**Tanulságok** -- 30+ futtatás tapasztalatai
5 perc	**Roadmap** -- Hová tartunk

1. A probléma

Miért nem elég promptolni?

AI kódolás ma -- a kihívások

Probléma	Tipikus megoldás	Eredmény
Divergencia	Prompt 2x futtatás = 2 különböző eredmény	Nem reprodukálható
Hallucináció	Ágensek kitalálják, amit nem tudnak	Hiányzó funkciók
Minőség-rulett	LLM ítéli meg a kód minőségét	Inkonzisztens
Spec drift	"Tesztek futnak" != "spec teljesül"	Részleges implementáció
Amnézia	Minden session nulláról indul	Ismétlődő hibák
Hibajavítás	Kézi debugging, órák	Elveszett idő

A legtöbb AI kódolóeszköz **nem-determinisztikus** -- ugyanaz a prompt, más eredmény.
A SET ezt mérnöki problémaként kezeli.

A SET megközelítése

Kihívás	SET megoldás	Eredmény
Divergencia	3 rétegű template rendszer	**83-87%** strukturális konvergencia
Hallucináció	OpenSpec workflow + elfogadási kritériumok	Spec ellen implementál
Minőség-rulett	Programozott gate-ek (exit kód, nem LLM ítélet)	Determinisztikus pass/fail
Spec drift	Coverage tracking + auto-replan	100% spec lefedettség
Amnézia	Hook-alapú memória (5 réteg)	100% kontextus megőrzés
Hibajavítás	Issue pipeline: detect -> investigate -> fix	**30 másodperc** recovery

> **"We don't prompt -- we specify."**

2. A bemenet

A spec minősége határozza meg az output minőségét

Egy jó spec felépítése

```
spec.md
├── Adatmodell          -- entitások, mezők, kapcsolatok, enumok
├── Oldalak             -- szekciók, oszlopok, komponensek
├── Design tokenek      -- hex színek, fontok, spacing értékek
├── Auth & szerepek     -- védett útvonalak, regisztráció
├── Seed data           -- valós nevekkel, nem "Termék 1"
├── i18n                -- lokálék, URL struktúra
└── Üzleti követelmények -- felhasználói történetek, elfogadási kritériumok
```

Te vagy a product owner, az ágensek a dev csapat, a spec a sprint backlog.

A különbség: ez a sprint órákat vesz igénybe, nem heteket.

Spec + Figma Design

```
$ head -35 tests/e2e/scaffolds/minishop/docs/v1-minishop.md

# MiniShop v1 — Webshop Feature Spec

> Next.js 14+ App Router webshop with Prisma (SQLite), shadcn/ui, Tailwind CSS, NextAuth.js v5

## Design

**Figma Design:** https://www.figma.com/make/9PH3uS4vWjSj6cUPhTGZSt/set-minishop?p=f&t=zvhTdumJeYUpKrJm-0

**Local design snapshot:** `docs/figma-raw/9PH3uS4vWjSj6cUPhTGZSt/` — pre-fetched via `set-figma-fetch`, contains

## Starting Point

There is NO application code in the scaffold. Agents create everything from scratch.

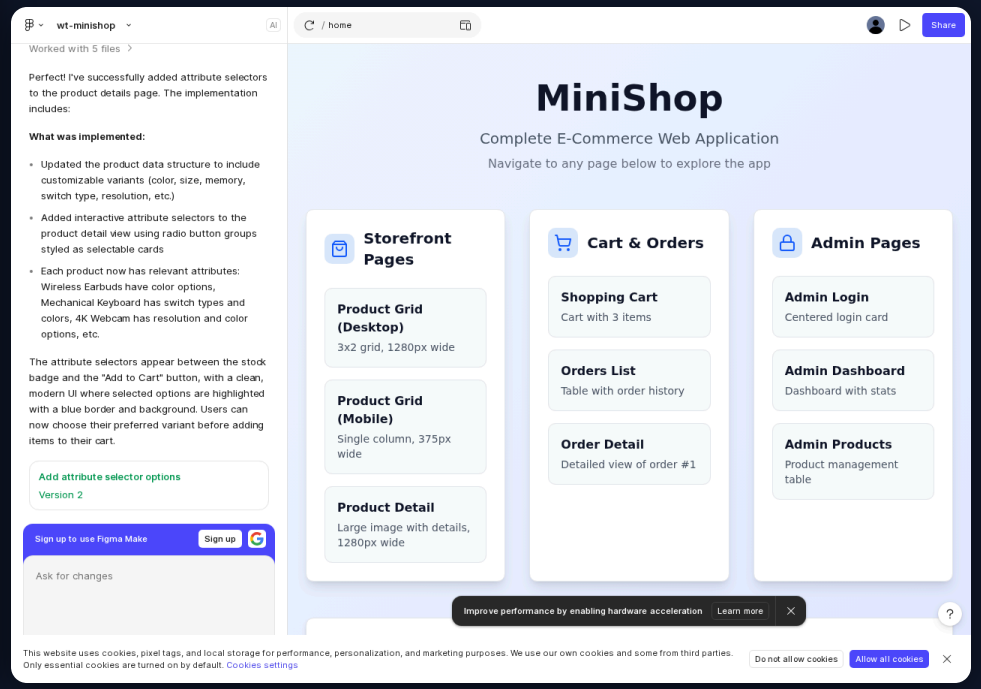
Platform configs are deployed by `set-project init --project-type web --template nextjs` before orchestration sta

**Setup (done by `run.sh` before orchestration):**
1. Copy this spec to `docs/v1-minishop.md`
2. `git init && set-project init --project-type web --template nextjs`
3. Orchestration starts — agents create everything from this spec

## Dependencies (package.json)

Agents must create `package.json` with these dependencies:

```jsonc
{
  "name": "minishop",
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "test": "vitest run",
    "test:e2e": "playwright test"
  },
```
```



Bal: Markdown spec -- az üzleti követelmények

Jobb: Figma Make design -- a vizuális terv

3. E2E Demo: MiniShop

Egy webshop felépítése specifikációból -- élőben

MiniShop -- a feladat

Egy **Next.js 14 e-commerce alkalmazás** -- teljes egészében spec-ből építve:

- Terméklistázás és részletes termékoldal
- Kosár funkció (hozzáadás, törlés, mennyiség)
- Pénztár és rendelés kezelés
- Admin autentikáció (bejelentkezés, regisztráció)
- Admin CRUD (termékek kezelése)
- Seeded adatbázis valós adatokkal

Kérdés: Mennyi idő alatt, hány hibával, mennyi emberi beavatkozással?

A pipeline

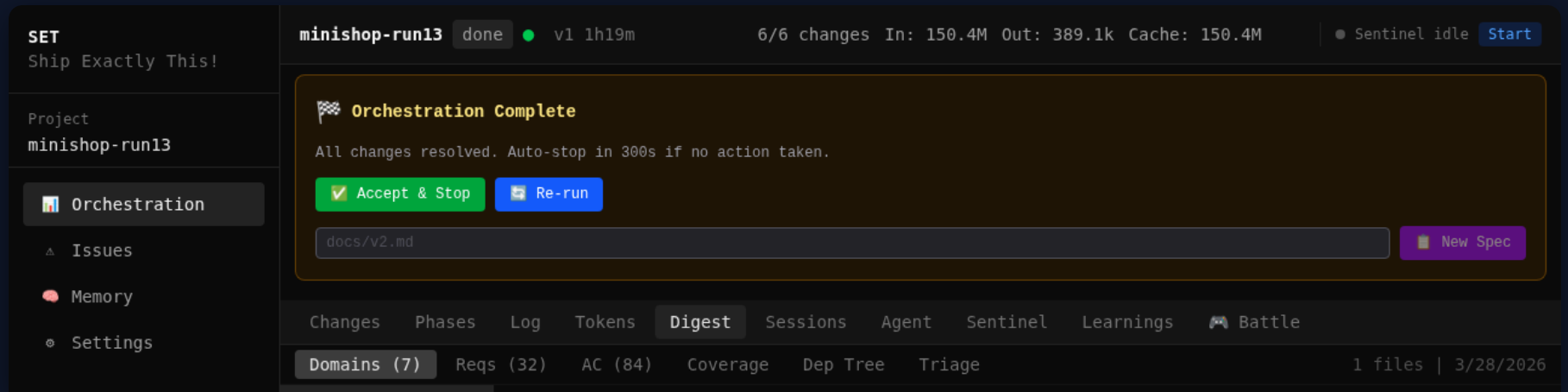
```
spec.md ──> digest ──> decompose ──> parallel agents ──> verify ──> merge ──> done
```

```
spec.md + design-snapshot.md (Figma)
  │
  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ Sentinel (autonóm szupervizor)                                           │
│  ├─ digest: spec → követelmények + domain összefoglalók                  │
│  ├─ decompose: független change-ek (DAG)                                 │
│  ├─ dispatch: git worktree-kbe párhuzamosan                              │
│  ├─ monitor: 15s polling, crash recovery                                 │
│  ├─ verify: quality gate-ek minden change-re                             │
│  ├─ merge: FF-only + post-merge smoke                                    │
│  └─ replan: amíg a spec 100%-ban lefedett                                │
└──────────────────────────────────────────────────────────────────────────┘
```

1. lépés: Digest -- Követelmények kinyerése

A spec-ből a digest modul kinyeri:

- **Domaineket** (pl. Products, Cart, Auth, Admin)
- **Követelményeket** domain-enként (pl. 32 db)
- **Elfogadási kritériumokat** (pl. 84 db WHEN/THEN)



2. lépés: Decompose -- Független change-ek

A planner a követelményekből **függetlenül implementálható change-eket** hoz létre:

Change	Függőség	Méret
`project-infrastructure`	--	Alapstruktúra, konfig
`products-page`	infrastructure	Terméklista + részletek
`cart-feature`	products	Kosár funkció
`admin-auth`	infrastructure	Admin bejelentkezés
`orders-checkout`	cart, admin-auth	Rendelés + pénztár
`admin-products`	admin-auth, products	Admin CRUD

Dependency DAG → fázisokra bontva:

3. lépés: Dispatch -- Párhuzamos ágensek

Minden change saját **git worktree**-t kap -- izolált fejlesztési környezet:

```
main/
├── .worktrees/
│   ├── project-infrastructure/   ← Agent 1
│   ├── products-page/            ← Agent 2
│   ├── admin-auth/               ← Agent 3 (párhuzamos a 2-vel!)
│   ├── cart-feature/             ← Agent 4
│   ├── orders-checkout/          ← Agent 5
│   └── admin-products/           ← Agent 6
```

Minden ágensben a **Ralph Loop** fut:

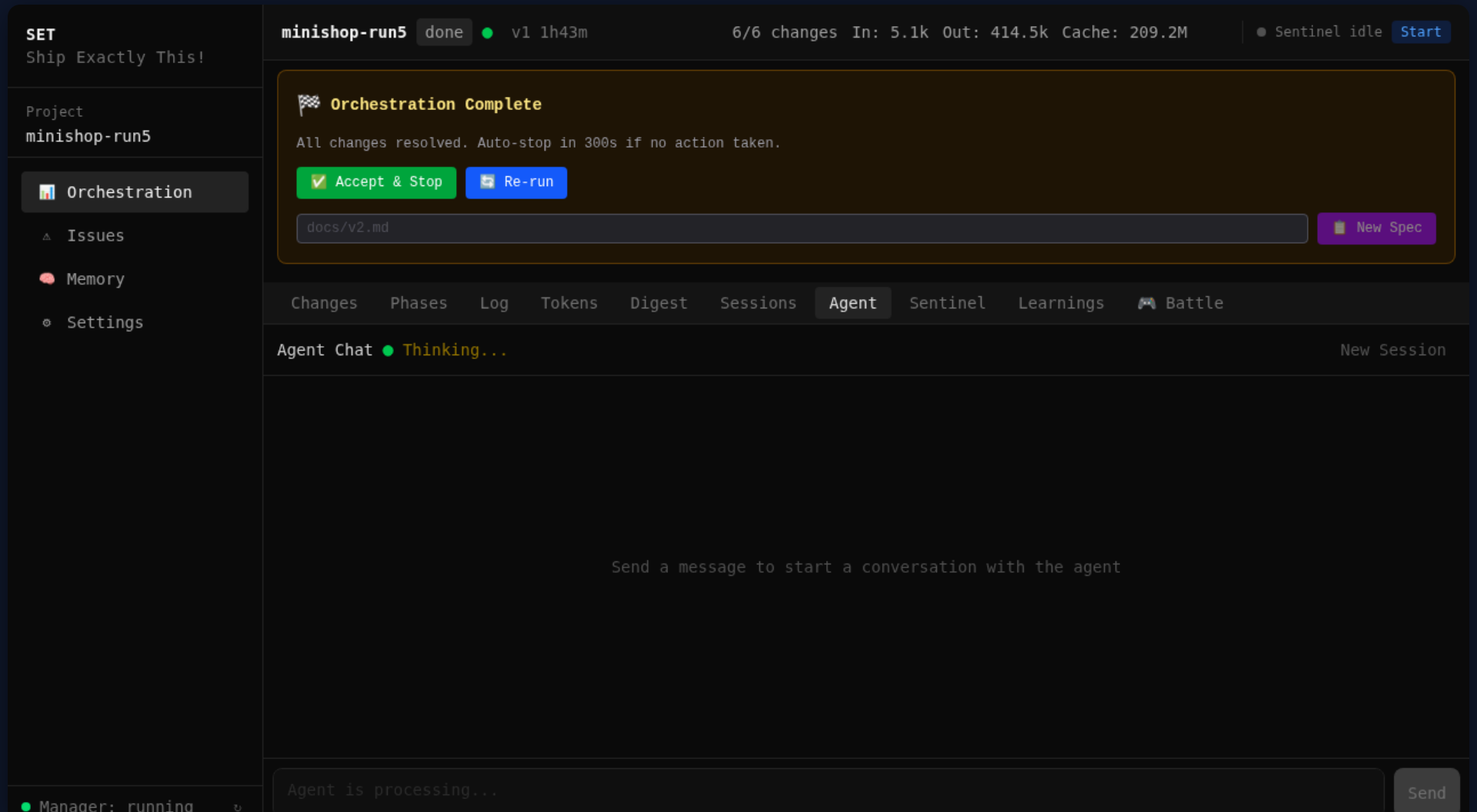
```
proposal → design → spec → tasks → implementáció → tesztek → kész
```

4. lépés: Monitor -- Valós idejű felügyelet

A Sentinel **15 másodpercenként** poll-ozza az összes ágenst:

- **Haladás követés** -- hol tart minden change?
- **Stall detekció** -- ha >120s nincs előrelépés → vizsgálat
- **Crash recovery** -- PID eltűnt? → diagnózis → újraindítás
- **Token tracking** -- költség figyelése change-enként

Közben az ágens dolgozik...



5. lépés: Verify -- Quality Gates

Minden change **6 gate-en** megy keresztül merge előtt:

```
Jest/Vitest (8s) → Build (35s) → Playwright E2E (45s)
    → Code Review (25s) → Spec Coverage → Post-merge Smoke (15s)
```

Gate	Idő	Mit ellenőriz	Típus
Test	8s	Unit/integration tesztek	Determinisztikus (exit code)
Build	35s	Type check + bundle	Determinisztikus (exit code)
E2E	45s	Böngésző-alapú tesztek	Determinisztikus (exit code)
Review	25s	Kód minőség	LLM (CRITICAL = fail)
Spec Coverage	--	Követelmény lefedettség	LLM + pattern match
Smoke	15s	Post-merge sanity check	Determinisztikus

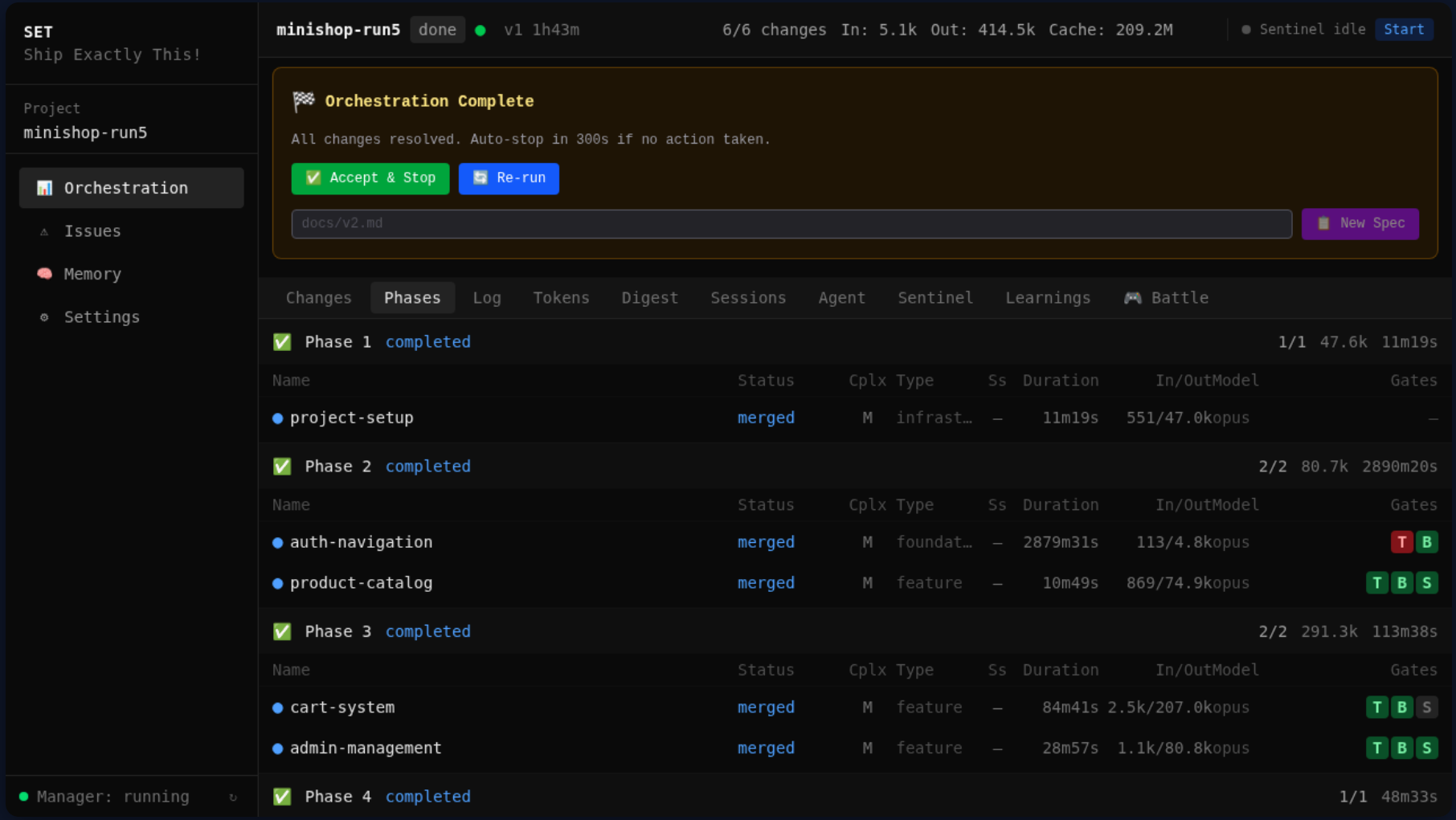
Self-Healing: 5 gate hiba, 5 automatikus javítás

A MiniShop futtatásban **5 gate hiba** volt -- mind automatikusan javítva:

#	Hiba	Gate	Javítás
1	Hiányzó teszt fájl	Test	Agent hozzáadta a teszt fájlt
2	Jest config hiba	Build	Agent javította a path mapping-et
3	Playwright auth teszt (3 spec)	E2E	Agent frissítette a redirect-eket
4	Post-merge type error	Build	Agent szinkronizált main-nel
5	Kosár teszt race condition	E2E	Agent hozzáadott `waitForSelector`-t

> Gate-ek nélkül ez az **5 hiba bekerült volna main-be** és kaszkád hibákat okozott

6. lépés: Merge -- Integráció



7. lépés: Replan -- 100% lefedettség

```
                    ┌──── coverage < 100%? ────┐
                    │                          │
merge ──▶ coverage check ──▶ done          replan
                                              │
                                     decompose újra
                                     a hiányzó REQ-ekre
```

A replan gate ellenőrzi:

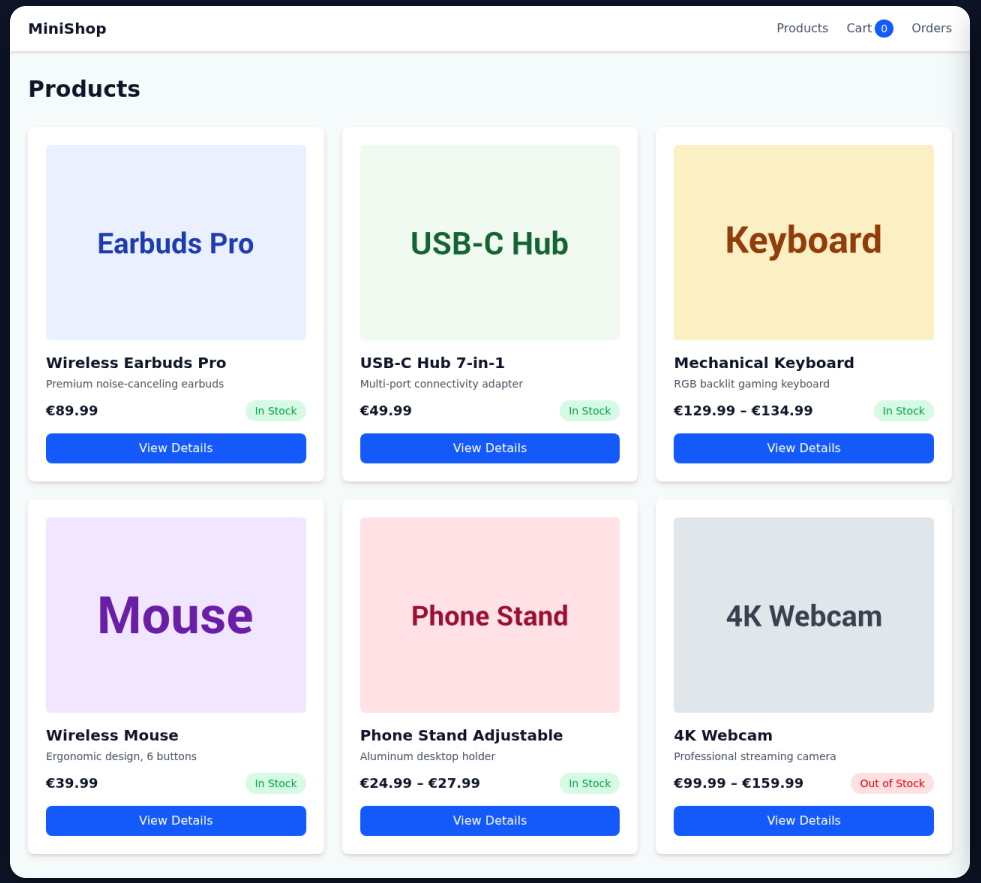
- Minden `[REQ: ...]` megvalósult-e?
- Van-e lefedetlen domain?
- Ha igen: **új change-ek automatikus tervezése** a hiányokra

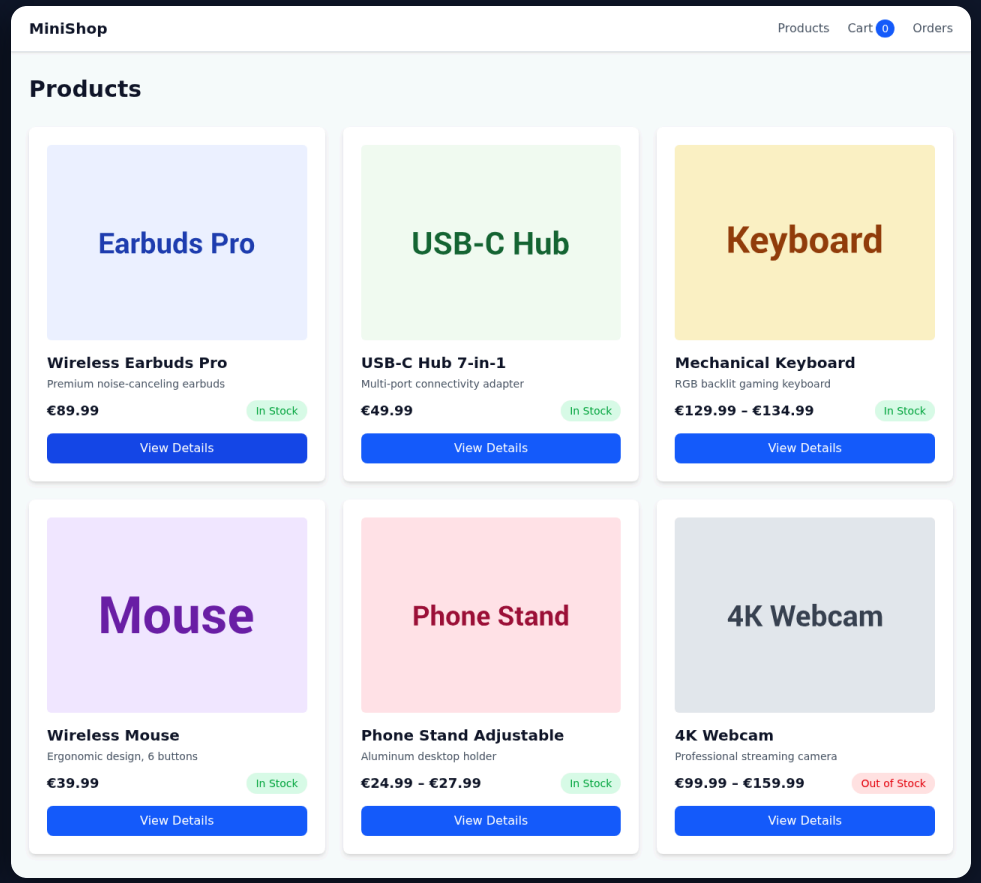
> A rendszer nem áll le amíg a spec 100%-ban nincs lefedve.

Az eredmény

Amit a pipeline produkált

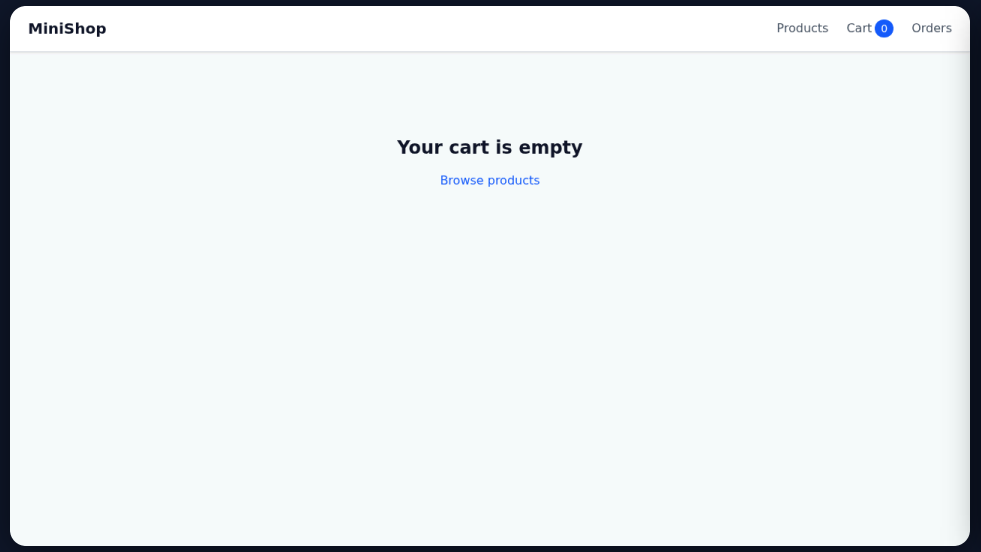
MiniShop -- a kész alkalmazás

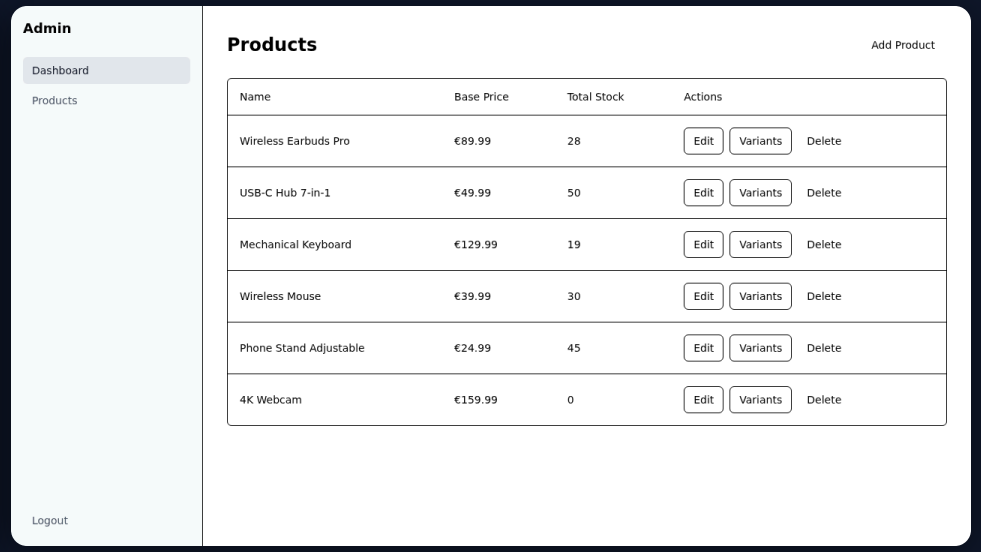




Terméklistázás és **termék részletek** -- valós adatokkal, működő navigációval.

MiniShop -- Kosár és Admin

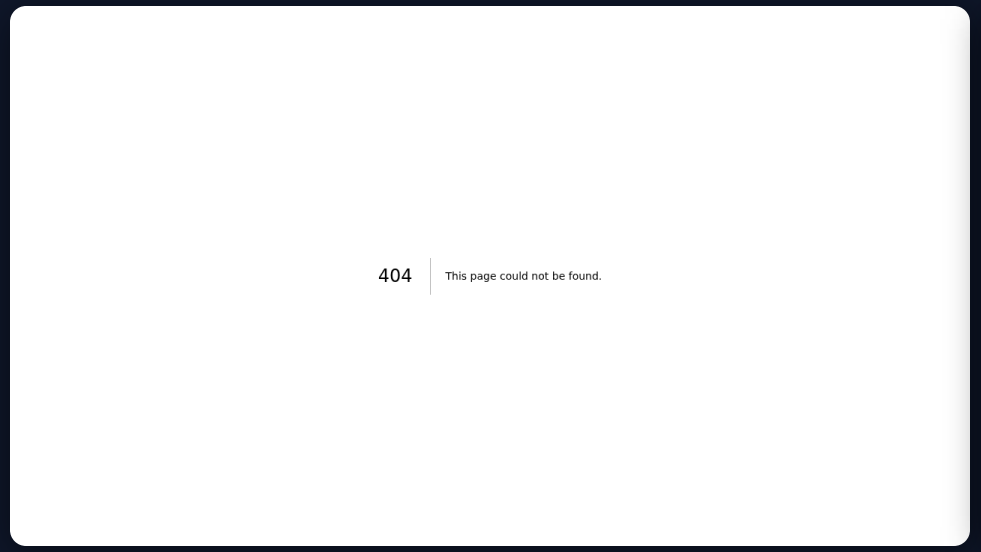


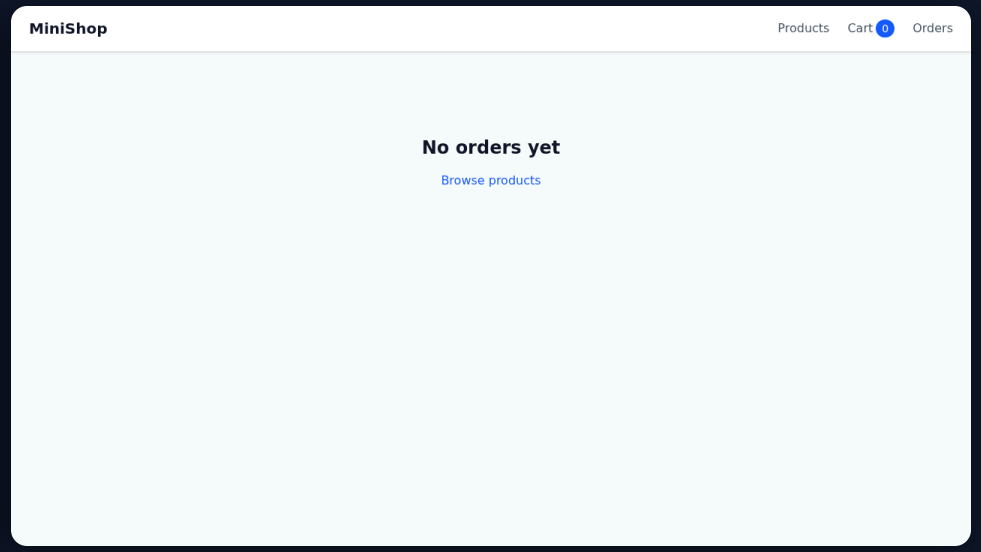


Bal: Működő kosár -- mennyiség módosítás, összesítés

Jobb: Admin dashboard -- védett route, session-alapú auth

MiniShop -- Admin CRUD és rendelések





Bal: Termék CRUD (létrehozás, szerkesztés, törlés)

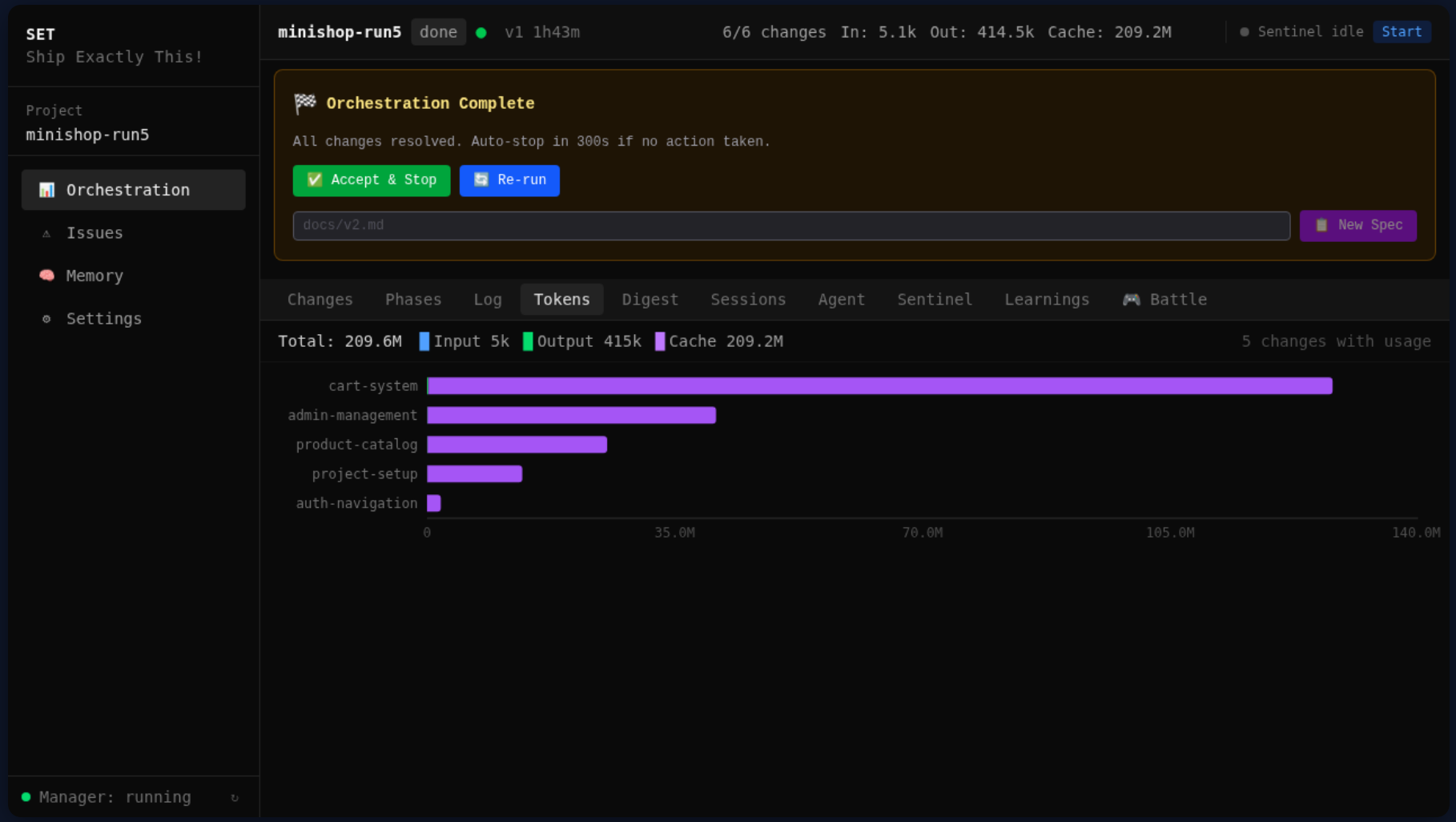
Jobb: Rendelés kezelés -- státuszok, szűrés

> Minden oldal **működő adatbázissal**, **funkcionális navigációval** és **reszponzív**

A számok

Metrika	Érték
Tervezett change-ek	**6**
Sikeresen mergelt	**6/6 (100%)**
Össz futási idő	**1 óra 45 perc**
Aktív build idő	~1 óra 25 perc
Emberi beavatkozás	**0**
Merge konfliktus	**0**
Jest unit tesztek	38 (6 suite)
Playwright E2E tesztek	32 (6 spec fájl)
Git commitok	39
Össz token	2.7M

Token felhasználás



4. Quality Gates

Determinisztikus minőségbiztosítás

Miért nem elég az LLM code review?

Korai kísérletek (CraftBrew run #3):

- LLM-alapú code review gate
- Ágensek "gaming"-elték a reviewer-t (részletes kommentek = jó kód)
- Átment egy `TODO: implement later` ami eltörte a checkout-ot
- Inkonzisztens pass/fail döntések

A megoldás:

- **Test, Build, E2E** = exit kód 0 vagy nem = **determinisztikus**
- Egy elbukó Jest teszt egyértelmű -- nem lehet "meggyőzni"
- Egy `next build` ami 1-es exit kóddal tér vissza nem hazudik
- Csak a spec coverage maradt LLM-alapú (explicit PASS/FAIL parsing)

Gate Profiles -- testreszabhatóság

Nem minden change-hez kell minden gate:

Change típus	Test	Build	E2E	Review	Coverage
Feature	HARD	HARD	HARD	HARD	HARD
Bugfix	HARD	HARD	SOFT	HARD	SOFT
Cleanup	SOFT	HARD	SKIP	SOFT	SKIP
Config	SKIP	HARD	SKIP	SOFT	SKIP

- **HARD** -- fail = block merge
- **SOFT** -- fail = warning, nem block
- **SKIP** -- nem fut

> A profile rendszer a `ProjectType` ABC-n keresztül bővíthető

Playwright E2E tesztek

Search tests

All 54 | Passed 47 | Failed 7 | Flaky 0 | Skipped 7

3/28/2026, 1:15:50 PM Total time: 10.4s

app-screenshots.spec.ts

✕	**Consumer app screenshots › app — home** app-screenshots.spec.ts:125	639ms
✕	**Consumer app screenshots › app — products** app-screenshots.spec.ts:125	597ms
✕	**Consumer app screenshots › app — product-detail** app-screenshots.spec.ts:125	167ms
✕	**Consumer app screenshots › app — cart** app-screenshots.spec.ts:125	384ms
✕	**Consumer app screenshots › app — admin-login** app-screenshots.spec.ts:125	500ms
✕	**Consumer app screenshots › app — admin-dashboard** app-screenshots.spec.ts:125	506ms
✕	**Consumer app screenshots › app — admin-products** app-screenshots.spec.ts:125	476ms

change-actions.spec.ts

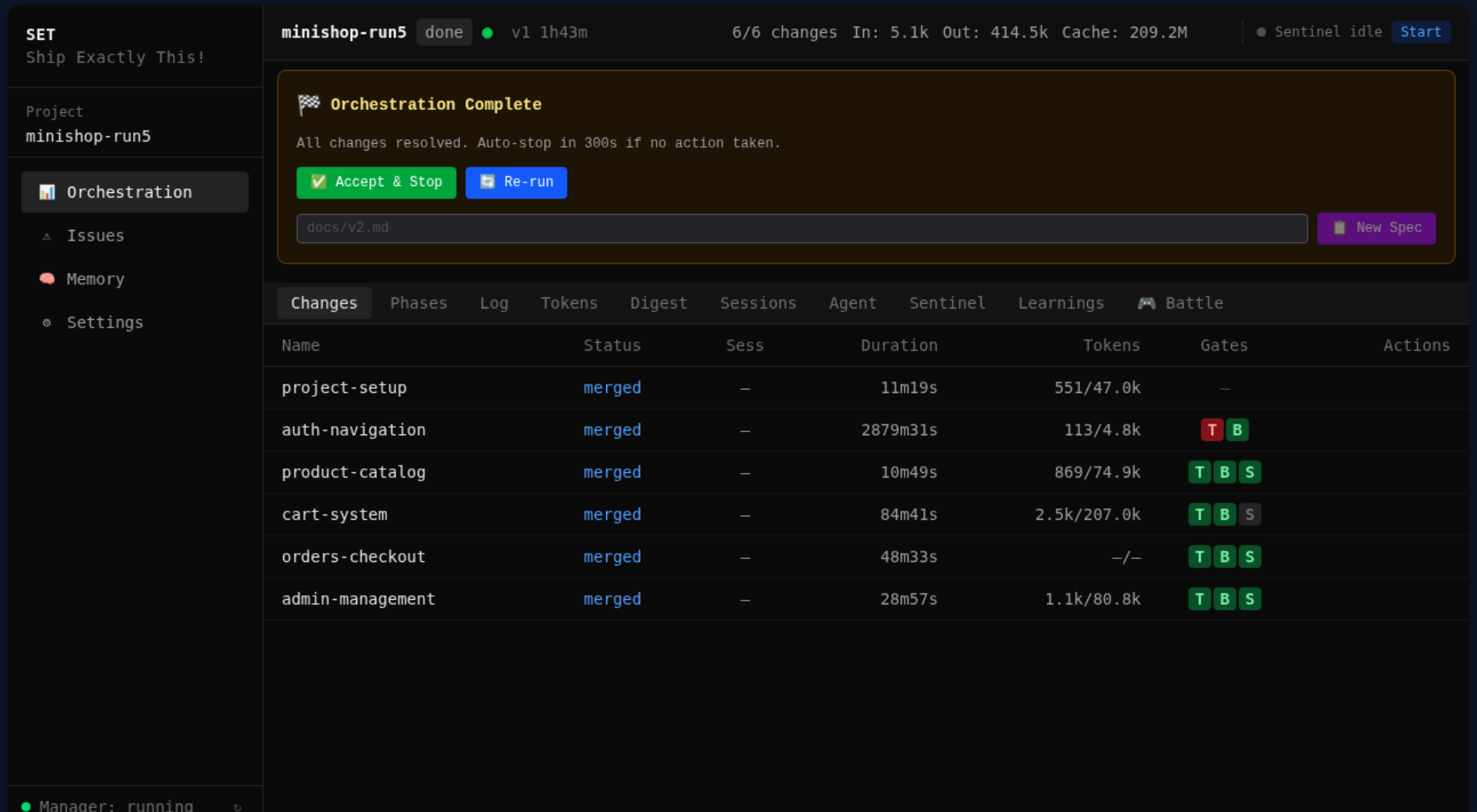
✓	**merged change has no action buttons** change-actions.spec.ts:8	1.1s

changes-data.spec.ts

5. Dashboard & Monitoring

Valós idejű rálátás mindenre

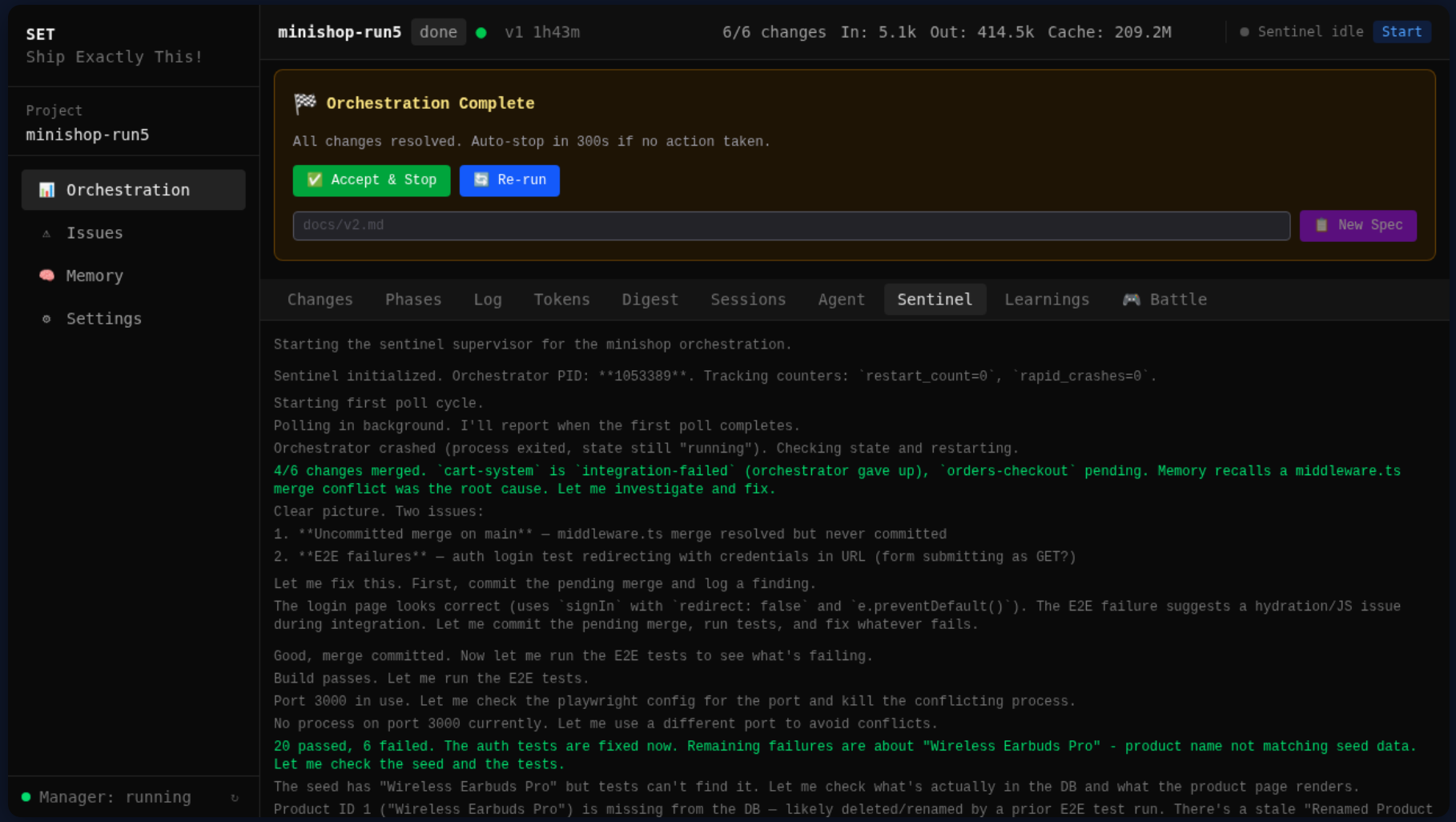
Web Dashboard -- Áttekintés



Dashboard tabok

Tab	Mit mutat
Changes	Összes change státusza, fázis, merge sorrend
Phases	Függőségi fa, gate badge-ek (B/T/E/R/V)
Tokens	Per-change token felhasználás chart
Sessions	Agent session lista, időtartam, tokenek
Sentinel	Szupervizor döntések, restart, stall
Log	Nyers orkesztrátor output, kereshető
Agent	Élő agent terminál
Learnings	Tanulságok, gate hibák
Digest	Követelmények + lefedettség

Sentinel felügyelet



6. Architektúra

3 réteg, bővíthető plugin rendszer

3 rétegű architektúra

```
┌──────────────────────────────────────────────────────────────┐
│ Layer 3: External Plugins (külön repók)                      │
│  └─ pl. set-project-fintech (IDOR, PCI compliance)           │
├──────────────────────────────────────────────────────────────┤
│ Layer 2: Built-in Modules (modules/)                         │
│  ├─ web/     → WebProjectType: Next.js, Prisma, Playwright   │
│  └─ example/ → DungeonProjectType: referencia implementáció  │
├──────────────────────────────────────────────────────────────┤
│ Layer 1: Core (lib/set_orch/)                                │
│  ├─ engine.py      → Állapotgép, orkesztrálás                │
│  ├─ dispatcher.py  → Worktree létrehozás, agent dispatch     │
│  ├─ merger.py      → Merge queue, integration gate-ek        │
│  ├─ verifier.py    → Quality gate-ek (test/build/E2E/...)    │
│  ├─ planner.py     → Spec dekompozíció, DAG generálás        │
│  └─ digest.py      → Követelmény kinyerés                    │
└──────────────────────────────────────────────────────────────┘
```

Projekt méret

```
set-core/
├── lib/set_orch/     59K sor Python     ← Core engine
├── modules/web/       5K sor Python     ← Web plugin
├── mcp-server/       30K sor            ← MCP szerver
├── web/              14K sor TypeScript  ← Dashboard
├── bin/              15K sor Shell       ← 57 CLI eszköz
├── openspec/specs/   23K sor            ← 376 specifikáció
├── docs/             22K sor            ← Dokumentáció
└── templates/core/    3K sor            ← Deploy-olt szabályok
                     ─────
                     134K LOC összesen
```

Statisztika	Érték
Fejlesztési idő	950+ óra
Commitok	1,295 (~16/nap)

7. Tanulságok

30+ futtatás, valós production tapasztalatok

8 tanulság az élesből

1. Ágenseknek struktúra kell, nem prompt

- OpenSpec artifacts (proposal → spec → tasks) tartja a fókuszt
- Nélküle: 3 ágens, 3 különböző táblázat-könyvtár

2. Quality gate-eknek determinisztikusnak kell lenni

- Exit kód > LLM ítélet
- LLM review-t gaming-elték az ágensek

3. Merge konfliktusok a #1 kaszkád hiba

- Fázis-rendezés + dependency DAG + szekvenciális merge queue
- Cross-cutting fájlok: Prisma schema, i18n bundle, middleware

8 tanulság (folytatás)

5. E2E tesztelés feltár amit unit test nem

- Stale lock fájlok, zombie worktree-k, race condition a poll ciklusban
- Token counter overflow 10M+ cache token-nél

6. Stall detekció grace period kell

- `pnpm install` >60s stdout nélkül → watchdog megölte
- Kontextus-tudatos timeout: install=120s, codegen=90s, MCP=60s

7. A Sentinel megtérül

- 3 éjszakai futtatás elveszett crash miatt a sentinel előtt
- 5-10 LLM hívás/futtatás, órákat spórol

Skálázás: MiniShop vs CraftBrew

Metrika	MiniShop	CraftBrew #7	Szorzó
Change-ek	6	15	2.5x
Forrás fájlok	47	150+	3x
DB modellek	~8	28	3.5x
Merge konfliktus	0	4 (mind resolved)	--
Emberi beavatkozás	0	0	--
Futási idő	1h 45m	~6h	3.4x
Tokenek	2.7M	~11M	**4x**

> A token skálázás **szuperlineáris** (4x token 2.5x change-re) -- a későbbi change-eknek

Konvergencia -- reprodukálhatóság mérése

Két független MiniShop futtatás összehasonlítása:

Dimenzió	Egyezés
DB séma (modellek, mezők, relációk)	**100%**
Konvenciók (naming, struktúra)	**100%**
Route-ok (URL-ek, API végpontok)	**83%**
Összesített	**83/100**

A 83%-os score kontextusa:

- A maradék 17% **stilisztikai**, nem strukturális

8. Roadmap

Hová tartunk

Fejlesztési irányok

Irány	Cél	Státusz
Divergencia csökkentés	Template optimalizálás, scaffold tesztelés	Egyszerű projekteknél mérhető javulás
Build idő optimalizálás	Párhuzamos gate-ek, inkrementális build, cache	Jelenleg szekvenciális; kutatás alatt
Session context reuse	Kontextus újrahasznosítás iterációk között	Cold-start token overhead csökkentés
Memória optimalizálás	Relevancia scoring, dedup, auto-rule konverzió	Dedup + konszolidáció működik
Gate intelligence	Adaptív threshold-ok múltbeli pass rate-ek alapján	Gate profile-ok működnek
Merge conflict	Proaktív cross-cutting fájl konfliktus	Fázis rendezés működik

Nagyobb tervek

Core/Web szétválasztás

- 170+ web-specifikus referencia szivárgott a core-ba
- Áthelyezés `modules/web/`-be új `ProjectType` ABC metódusokon keresztül
- Cél: külső pluginek ne függjenek web logikától

NIS2 Compliance Layer

- EU 2022/2555 irányelv támogatás
- Template rule-ok, verifikációs szabályok, dedikált gate
- Opt-in konfigurációs flag-gel

shadcn/ui Design Connector

- `components.json`, `tailwind.config.ts`, `globals.css` parsing

A teljes ökoszisztéma

Repository	Leírás
set-core	Core engine, web modul, dashboard, CLI
set-spec-capture	Spec kinyerés webből, PDF-ből, beszélgetésből
set-voice-agent-delivery	Hang-alapú agent delivery
set-project-example	Referencia plugin (Dungeon Builder)

Technológiai stack:

Komponens	Technológia
Agent runtime	Claude Code (Anthropic)
Workflow	OpenSpec
Izoláció	Git worktree-k

Összefoglalás

Spec → Digest → Decompose → Dispatch → Verify → Merge → Done

6 change | 1h 45m | 0 beavatkozás | 70 teszt | 100% spec coverage

Kérdések?

Források:

- Repo: `git.setcode.dev/root/set-core`
- Web: `setcode.dev`
- Benchmarks: `docs/learn/benchmarks.md`
- Lessons learned: `docs/learn/lessons-learned.md`

Próbáld ki:

```
# Telepítés
pip install -e .
pip install -e modules/web

# Első futtatás
./tests/e2e/runners/run-micro-web.sh
```